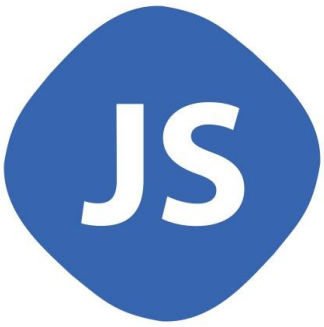


Параметры и возврат значений

- Параметры функций
- Возврат значений

Параметры функций

Функции можно передать любую информацию, используя **параметры**. Параметры представляют способ передачи в функцию данных. Параметры указываются в скобках после названия функции. Если функция принимает несколько параметров, то они **перечисляются через запятую**.

Значение, передаваемое в качестве параметра функции, также называется **аргументом**.

Если быть более точным:

- **Параметр** – это переменная, указанная в круглых скобках в **объявлении** функции.
- **Аргумент** – это значение, которое передаётся функции при её **вызове**.

Мы объявляем функцию со списком параметров, затем вызываем ее, передавая список аргументов. В приведённом примере функция объявляется с двумя **параметрами**: user и role, и вызывается с двумя **аргументами** 'Евгений' и 'Администратор'.

```
// параметры: user, role

function showUser(user, role) {

    let msg = user + ' (' + role + ') ';

    alert(msg);
```



```
}
// аргументы: 'Евгений', 'Администратор'
showUser('Евгений', 'Администратор');
```

Когда функция вызывается, переданные значения параметров копируются в **локальные переменные user** и **role**. Затем они используются в теле функции.

> ***Еще раз***. В приведённом примере можно сказать: функция showUser() объявляется с двумя **параметрами** (user, role), затем вызывается с двумя **аргументами** ('Евгений', 'Администратор').

Параметры можно именовать, как угодно, их имена имеют смысл исключительно в **теле функции**. При наименовании параметров применяются те же правила, что и для переменных.

Особенности использования параметров (аргументов) функций JavaScript:

- Параметры, которые передаются функции, могут передаваться как по **значению**, так и по **ссылке**.
- Определения функций JavaScript не указывают **типы данных** для параметров.
- Функции JavaScript не выполняют **проверку типа** на передаваемых аргументах.
- Функции JavaScript не проверяют **количество полученных аргументов**.

example_1. Параметры функции

```
<script>
    // объявляем функцию
    function mult(num1, num2) {
        // тело функции
        let res = num1 * num2;
```

```
    return res;
}

// объявим переменную value
var value = 4;
// во внешний код вернем значение, возвращаемое функцией
// значение запишем в переменную
var x = mult(value, 6);
console.log(x); // 24
</script>
```

В функцию могут передаваться параметры **не только примитивных типов** (числа, строки, логический булев). Рассмотрим передачу в функцию в качестве параметров следующие типы и структуры данных:

- **массив**
- **объект**
- **коллекция**

Параметр функции – **массив**.

example_2. Параметр функции - массив

```
<script>
    // инициируем массив
    let auto = ["Buick", "Cadillac", "Chevrolet", "Chrysler",
    "Dodge", "Ford", "Pontiac", "Hummer"];
    // объявляем функцию
    // параметр функции - переменная arr
    function fnArray (arr) {
        // теперь arr - локальная переменная-массив внутри
        функции
```

```
        // инициализируем строку для вывода списка
        let line = "";
        // формируем список
        for (elem of arr) {
            line += `<li>${elem}</li>`;
        }
        // возвращаем строку, которая является списком
        return "<ul>" + line + "</ul>";
    }
    // вызов функции, аргумент - массив auto
    // вывод на экран результата вызова
    document.write (fnArray(auto));
</script>
```

Параметром функции может быть **объект**.

example_3. Параметр функции - объект

```
<script>
    // инициируем объект
    let tel =
    {
        firm: "DEXP",
        model: "H24F7000E",
        diag: "24",
        perm: "HD",
        matr: "LED",
```

```
        freq: "60 Гц",
        price: 25000,
        path: "H24F7000E.jpg",
        descr: "Телевизор LED DEXP H24F7000E отличается столь
        компактными размерами, что место для него найдется даже
        на самой маленькой кухне. Благодаря VESA-креплению
        разместить его получится даже на стене, чтобы освободить
        на горизонтальных поверхностях место для других вещей."
}
// объявляем функцию
// параметр функции - переменная obj
function fnObject(obj) {
    // теперь obj - локальная переменная-объект внутри
    функции
    // инициализируем строку для вывода
    let line = `
        Фирма: <b>${obj.firm}</b><br>
        Модель: <b>${obj.model}</b><br>
        Диагональ: <b>${obj.diag}</b><br>
        Разрешение экрана: <b>${obj.perm}</b><br>
        Матрица: <b>${obj.matr}</b><br>
        Количество кадров в секунду: <b>${obj.freq}</b><br>
        Цена: <b>${obj.price}</b><br>
        Изображение: <br><img src='${obj.path}'><br>
        Описание: <b>${obj.descr}</b><br>
    `;
    // возвращаем строку
```

```
        return line;
    }
    // вызов функции, аргумент - объект tel
    // вывод на экран результата вызова
    document.write (fnObject(tel));
</script>
```

Параметром функции может выступать **коллекция**.

example_4. Параметр функции - коллекция

```
<script>
    // инициируем коллекцию
    // выбираем коллекцию элементов p
    let citate = document.querySelectorAll("div#citate p");
    // объявляем функцию
    // параметр функции - переменная coll
    function fnCollection(coll) {
        // теперь coll - локальная переменная-коллекция внутри
        функции
        // инициализируем строку для вывода
        let line = "";
        for (el of coll) {
            line += `<li>${el.innerText}</li>`;
        }
        line = "<ol>" + line + "</ol>";
        // возвращаем строку
        return line;
```

```
    }
    // вызов функции, аргумент - коллекция citate
    // вывод на экран результата вызова
    document.write(fnCollection(citate));
</script>
```

Ну и, естественно, функция может принимать в качестве параметров набор **разнотипных переменных**.

example_5. Несколько разнотипных параметров

```
<script>
    // инициируем строку
    let str = "Тестируем параметры функции";
    // инициируем коллекцию
    let coll = document.querySelectorAll("ul li");
    // инициируем объект
    let obj = {login:"master", pwd:"12345"};
    // инициируем массив
    let arr = ["Buick", "Cadillac", "Chevrolet", "Chrysler"];
    // объявляем функцию
    // в качестве параметров - строка, коллекция, объект, массив
    function fnOutData (myStr, myCollection, myObj, myArray){
        console.log("Строка: ", myStr);
        console.log("Коллекция: ", myCollection);
        console.log("Объект: ", myObj);
        console.log("Массив: ", myArray);
    }
```

```
    // вызов функции
    fnOutData (str, coll, obj, arr);
</script>
```

Необязательные параметры

В отличие от ряда других языков программирования мы можем не передавать значения параметрам. Если параметр не указан, то его значением становится **undefined**.

Например, в коде демонстрационного примера **example_6** функция **showUser**(user, role) может быть вызвана с одним аргументом:

```
showUser('Евгений');
```

Это не приведёт к ошибке. В вызове не указан параметр **role**, поэтому предполагается, что **role == undefined**.

example_6. Необязательные параметры

```
<script>
    // объявляем функцию с двумя параметрами
    // параметры: user, role
    function showUser(user, role) {
        let msg = user + ' (' + role + ') ';
        alert(msg);
        console.log(msg);
    }
    // вызываем функцию с одним аргументом: 'Евгений'
    showUser('Евгений');
</script>
```



Параметры по умолчанию

Однако иногда бывает необходимо, чтобы параметры обязательно имели какие-то значения, например, **значения по умолчанию**. Если мы хотим задать параметру **role** значение по умолчанию, мы должны указать его значение при объявлении функции.

example_7. Параметры по умолчанию

```
<script>
    // объявляем функцию с двумя параметрами
    // параметры: user, role
    // параметр role задан по умолчанию
    function showUser(user, role = 'Модератор') {
        let msg = user + ' (' + role + ') ';
        alert(msg);
        console.log(msg);
    }
    // вызываем функцию с одним аргументом: 'Евгений'
    showUser('Евгений');
</script>
```

Теперь, если параметр **role** при вызове функции не указан, его значением будет строка "Модератор". Параметр (параметры) по умолчанию **должны быть последними** в списке параметров функции.

Остаточные параметры (Rest parameters)

Остаточные параметры или **неопределенное количество** параметров (или Rest-параметры).

Оператор "**...**" (многоточие) указывает, что с помощью параметра можно передать переменное количество значений.

example_8. Остаточные параметры

```
<script>
    // объявляем функцию с неопределенным количеством параметров
    function fnParamsDisplay(...params) {
        // params - массив
        console.dir("params - это массив:");
        console.dir(params);
        console.log("==========\n");
        console.log(params[0]);
        console.log(params[1]);
        console.log(params[2]);
        console.log(params[3]);
    }
    // вызов функции с 4 аргументами
    /*
    "99" - строка
    10 - число
    true - булев
    [{user:"sa"}, "php"] - массив из двух элементов (объект,
    строка)
    */
    fnParamsDisplay("99", 10, true, [{user:"sa"}, "php"]);
</script>
```

В данном случае параметр **...params** указывает, что вместо него можно передать неопределенное количество значений. В теле функции **params** фактически представляет **массив** переданных значений, которые мы можем получить.

Примечание. Вместо **params** вы можете использовать любое другое имя. Здесь важен оператор "**...**".

Если **params** – массив, то и работать с ним можно как с массивом.

example_9. Работа с остаточными параметрами в цикле

```
<script>
    // объявляем функцию с неопределенным количеством параметров
    function fnParamsDisplay(...vars) {
        // params - массив
        // массив можно перебрать в цикле
        for (value of vars){
            console.log(value);
        }
    }
    // вызов функции с 4 аргументами
    console.log ("-> вызов функции с 4 аргументами:");
    fnParamsDisplay("99", 10, true, [{user:"sa"}, "php"]);
    console.log ("==========\n");
    // вызов функции с 2 аргументами
    console.log ("-> вызов функции с 2 аргументами:");
    fnParamsDisplay("перестройка", "плоскогубцы");
</script>
```

Важно. Несмотря на то, что при вызове функции в нее передаются **отдельные значения**, в функции аргумент доступен в виде **массива**.



Можно сочетать параметры, сначала указать **явный**, потом - **неопределенное** количество параметров.

В демонстрационном примере **example_10** первый параметр – **строковый**, далее следует **неопределенное** количество параметров (параметр – params).

example_10. Сочетание явных и неявных параметров

```
<script>
    // объявляем функцию с двумя параметрами:
    /*
        1) str - явный параметр (строка)
        2) ...params - неопределенным количеством параметров
    */
    function display(str, ...params){
        console.log ("-> это первый параметр: ", str);
        console.log ("-> далее массив параметров: ");
        // params - массив
        for (value of params){
            console.log(value);
        }
    }
    // вызов функции
    display("html", "javascript", true, "css", 1);
    console.log ("==========\n");
    // вызов функции может быть и такой
    console.log ("вызов функции может быть и такой:\n");
    display();
</script>
```



Можно сделать так: первый параметр – **массив**, далее – **неопределенное** количество параметров.

example_11. Параметры: массив и неопределенное количество (тоже массив)

```
<script>
    // объявляем функцию с двумя параметрами:
    /*
        1) arr - явный параметр (массив)
        2) ...params - неопределенным количеством параметров
    */
    function display(arr, ...params){
        // первый параметр
        console.log("-> первый параметр:");
        console.dir(arr);
        // второй параметр
        console.log("-> второй параметр:");
        console.dir(params);
    }
    // вызов функции
    // первый параметр явный, массив - ["html", "javascript", "css"]
    // второй параметр неопределенный, массив из элементов - true, [1,2,3]
    display(["html", "javascript", "css"], true, [1,2,3] );
</script>
```

Не стоит путать неопределенное количество параметров и массив. В качестве неопределенного количества параметров могут передаваться массивы. Тогда внутри функции это **многомерный массив** (массив массивов).

Важно. Неопределенные параметры — это значения, объединенные в массив. Следовательно, массивы в качестве неопределенных параметров – многомерный массив.

example_12. Неопределенные параметры - массивы

```
<script>
    // инициируем первый массив
    let arr1 = [1,2,3];
    // инициируем второй массив
    let arr2 = [4,5,6];
    // объявляем функцию с неопределенным количеством параметров
    function display(...params){
        // контрольный вывод
        console.log(params);
        // params - массив массивов
        for (items of params){
            // items - элемент, который сам является массивом
            // а если это массив, то перебираем его в цикле
            for (val of items) {
                console.log(val);
            }
        }
    }
    // вызов функции
    display(arr1, arr2);
</script>
```

Любителям конспирологий. Если вы не разыскиваетесь за финансовые махинации и не являетесь злостным алиментщиком путать код таким образом стоит только при наличии *железных* причин. Полиция все равно *размотает*, а вот вы запутаетесь в таком коде точно. Код должен быть логичен и предсказуем.

Массив аргументов arguments

При необходимости мы можем получить все переданные параметры через глобально доступный специальный объект **arguments**.

Объект **arguments** функции является **псевдо-массивом**. Внутри функции вы можете ссылаться к аргументам следующим образом:

```
arguments[i];
```

где: i — это порядковый номер аргумента, отсчитывающийся с 0.

К первому аргументу, переданному функции, обращаются так: **arguments**[0]. Получить количество всех аргументов — **arguments**.**length**.

example_13. Массив аргументов arguments

```
<script>
    // объявляем функцию суммирования чисел
    function sum () {
        let result = 0;
        // arguments - специальный объект
        // console.log(arguments);
        // выполняем перебор аргументов циклом for
        for(i = 0; i < arguments.length; i++) {
            result += arguments[i];
```

```
        }
        // выводим результат в консоль
        console.log(result);
    }
    // вызываем функцию с разным количеством аргументов
    sum(5);  // 5
    sum(5, 5);  // 10
    sum(5, 5, 5);  // 15
</script>
```

При этом не важно, что при определении функции мы **не указали никаких параметров**, мы все равно можем их передать и получить их значения через массив **arguments**.

Объект **arguments** позволяет вызвать функцию, передавая в неё больше аргументов, чем формально объявлено. Это очень полезно, если вы не знаете точно, сколько аргументов будет принимать ваша функция. Вы можете использовать **arguments.length** для определения количества аргументов, переданных функции, а затем получить доступ к каждому аргументу, используя цикл по псевдо-массиву **arguments**.

Перебор элементов можно организовать и с помощью специального цикла **for...of**.

```
// выполняем перебор аргументов
for(let num of arguments) {
    result += num;
}
```

Функции в качестве параметров



Аргументы функции не ограничиваются простыми, сложными типами данных или коллекциями, на месте аргумента может быть **выражение**, которое бы вычислялось и присваивалось при отсутствии параметра.

В следующем демонстрационном примере функция принимает в качестве аргумента по умолчанию другую функцию.

example_14. Функция как аргумент

```
<script>
    // эта функция будет выступать в качестве аргумента
    function getRole() {
        // здесь выполняем какие-то действия, например, запрос к серверу
        // результат этих действий возвращаем во внешнюю программу
        let result = "Модератор";
        return result;
    }
    function showUser(user, role=getRole()) {
        // getRole() выполнится только если не передан параметр role
        // результат выполнения = значение role
        let msg = user + " (" + role + ")";
        alert(msg);
    }
    // первый вызов, 2 аргумента
    showUser("Евгений", "Администратор");
    // второй вызов, 1 аргумент
    // будет выполнена функция getRole()
```

```
    showUser("Евгений");
</script>
```

Строго говоря, более широкое применение получил способ передачи в функцию в качестве аргумента исходного кода другой функции.

example_15. Исходный код функции как аргумент

```
<script>
    // эта функция будет выступать в качестве аргумента
    function getRole() {
        // здесь выполняем какие-то действия, например, запрос к серверу
        // результат отправляем во внешний код
        let result = "Модератор";
        return result;
    }
    // функция двух параметров:
    // - user - строковый
    // - role - исходный код другой функции
    function showUser(user, role) {
        // исходный код с круглыми скобками запустит функцию-аргумент
        if (role){
            let msg = user + " (" + role() + ")";
            alert(msg);
        }
    }
    // вызов функции, 2 аргумента
```

```
    showUser("Евгений", getRole);
    // вызов функции, 1 аргумент
    showUser("NoName");
</script>
```

Примечание. Передача функций в качестве параметров широко применяемая технология написания функций. Рассмотрим эту возможность отдельным уроком.

Передача параметров по значению

Примитивные параметры (число, строка, логическое значение) передаются в функцию по **значению**. Параметр передаётся в функцию, но, если функция меняет значение параметра, это изменение **не отразится глобально в вызывающей функцию программе**.

Функция всегда получает только **копию значения**.

Рассмотрим, что это значит в практическом плане.

example_16. Передача параметра по значению

```
<script>
    // тестируем параметр по значению
    // объявляем функцию - инструкцию
    // в функцию передаем параметр-число x
    function fnChangeNum(x) {
        // изменили переменную x
        var x = x + 10;
        console.log("в функции изменили аргумент, x = ", x);
```

```
    }
    // объявили переменную, присвоили значение
    var n = 10;
    console.log("-> переменная n ДО вызова функции: n = ", n); //
    10
    // вызываем функцию
    fnChangeNum(n);  // 20
    console.log("-> переменная n ПОСЛЕ вызова функции: n = ", n);
    // 10
</script>
```

При вызове функции **fnChangeNum** параметром передаем число **n**. Функция получает параметр и прибавляет к нему число 10. Однако, после вызова функции число **n** не изменилось, хотя в самой функции произошло увеличение значения параметра.

Это произошло потому, что при вызове функция **fnChangeNum** получает **копию значения переменной n**. И любые изменения с этой копией никак не затрагивают саму переменную **n**.

> ***Важно***. Аргументы примитивных параметров передаются в функцию **по значению**. Если функция изменяет значение аргумента, это изменение **не отражается** на глобальном состоянии этой переменной.

Передача параметров по ссылке

В отличии от примитивных параметров **объекты** и **массивы** передаются **по ссылке**. То есть функция получает **сам объект или массив**, а не их копию.

example_17. Параметр массив

```
<script>
```

```
    // тестируем параметр по ссылке
    // объявляем функцию - выражение
    // в функцию передаем параметр-массив arr
    fnChangeArr = function (arr) {
        // меняем первый элемент массива
        arr[0] = "javaScript";
    }
    // инициируем массив исходными данными
    var numbers = [1, 2, 3];
    console.log("исходный массив: ", numbers);  // [1, 2, 3]
    // вызываем функцию
    fnChangeArr (numbers);
    console.log("массив после вызова функции: ", numbers);   //
    ["javascript", 2, 3]
</script>
```

example_18. Параметр объект

```
<script>
    // тестируем параметр по ссылке
    // объявляем стрелочную функцию
    // в функцию передаем параметр-объект obj
    fnChangeObj =  obj => {
        // меняем свойство объекта
        obj.role = "moderator";
    }
    // инициируем объект исходными данными
```

```
    var obj = {login: "master", role:"user",
    email:"master@mail.ru"};
    console.log("-> исходный объект: ", obj);
    // вызываем функцию
    fnChangeObj(obj);
    console.log("-> объект после вызова функции: ", obj);
</script>
```

Возврат значения

Функция может вернуть результат, который будет передан в вызвавший её код (внешний код).

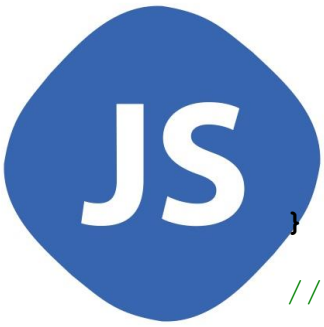
Простейшим примером может служить функция сложения двух чисел:

```
function sum(a, b) {
    return a + b;
}
let result = sum(1, 2);
alert(result); // 3
```

Директива **return** может находиться **в любом месте** тела функции. Как только выполнение доходит до этого места, функция **останавливается**, и значение **возвращается** в вызвавший её код.

Директиву **return** можно использовать **без значения**. Это приведёт к немедленному **выходу** из функции:

```
function checkUser(){
    // ...
    if (login != "admin") {
```



```
            return;
        }
        // ...
}
```

Вызовов **return** может быть несколько.

example_19. Несколько директив return

```
<script>
    // объявляем анонимную функцию
    let checkUser = function() {
        // получение данных от пользователя
        let login = prompt('Введите логин: ');
        let pwd = prompt('Введите пароль: ');
        // анализ данных
        if (login == "admin" && pwd == "12345") {
            return true;
        }
        else {
            return false;
        }
    }
    // принятие решения о доступе
    if (checkUser()) {
        document.write ('<h1>Добро пожаловать на сайт!</h1>');
    } else {
        alert('Пользователь с такими данными в системе не
        обнаружен.');
```

```
        document.write ('<h1>Доступ к ресурсу закрыт :(</h1>');
    }
</script>
```

Функция может возвращать любой из типов данных, например **массив**.

example_20. Возврат во внешний код массива

```
<script>
    // объявляем стрелочную функцию
    let fnReturnArray = () => {
        // функция возвращает массив
        return ["балалайка", "медведь", "матрешка"];
    }
    // вызов функции
    let arr = fnReturnArray();
    // вывод массива в консоль
    console.log(arr);
</script>
```

В следующем примере функция возвратит во внешний код сформированный **объект**.

example_21. Возврат во внешний код объекта

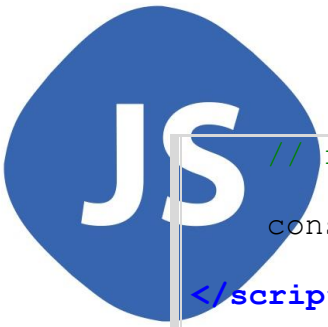
```
<script>
    // объявляем функцию-выражение
    let fnReturnObj  = function (...params) {
        // формируем объект
        let obj = {
            name: params[0],
```

```
            login: params[1]
        };
        // функция возвращает объект
        return obj;
    }
    // получение данных от пользователя
    let name = prompt('Введите Имя: ');
    let login = prompt('Введите Логин: ');
    // вызов функции
    let obj = fnReturnObj(name, login);
    // вывод объекта в консоль
    console.log(obj);
</script>
```

Во внешний код можно возвратить **коллекцию**.

example_22. Возврат во внешний код коллекции

```
<script>
    // объявляем функцию-инструкцию
    function fnReturnCollection () {
        // получаем коллекцию узлов DOM
        let coll = document.querySelectorAll("ul li");
        // функция возвращает коллекцию
        return coll;
    }
    // вызов функции
    let coll = fnReturnCollection();
```



```
    // вывод коллекции в консоль
    console.log(coll);
</script>
```

Разделение функций на источники и потребители

Итак, мы выяснили, что функция может (и даже должна ;):

- принимать в качестве **аргумента** все, что угодно (почти);
- возвращать в качестве **результата** все, что угодно (ну, тоже почти).

В предыдущих примерах наши функции **получали** данные и **выводили** их, например, в браузер. Такое поведение вполне себе возможно, но в реальных приложениях логику работы приложения желательно разбивать не только на функции, но и на **действия**. Таким образом, в идеале, функция должна выполнять одно действие. Это значит, что одна функция выполняет доступ к данным (такие функции иногда называют источниками), вторая функция выполняет обработку этих данных, например их вывод (функции потребители).

Используя возможности функций по приему и возврату данных реализовать такое поведение функций не составит труда.

В коде демонстрационного примера **example_23** создадим две функции. Первая функция будет выполнять поиск в подключаемом массиве данных и возвращать во внешний код данные согласно некоторому критерию. Это функция **источник** данных. Результат работы функции-источника будет подаваться на вход второй функции для форматного вывода в браузер. Это функция **потребитель**.

example_23. Определение функций

```
<script>
    // функция поиска курса по идентификатору
    function fnGetCourse(id) {
        for (let course of courses) {
```

```
            if (course.id == id)
                return course;
        }
    }
    // функция форматного вывода данных о курсе
    function fnOutCourse(obj) {
        // формируем вывод
        let out = `
            Идентификатор курса: <b>${obj.id}</b><br>
            Название: <b>${obj.name}</b><br>
            Срок обучения: <b>${obj.time}</b><br>
            Расписание: <b>${obj.shedule}</b><br>
            Уровень: <b>${obj.level}</b><br>
            Преподаватель: <b>${obj.teacher}</b><br>
            Цена: <b>${obj.price}</b><br>
        `;
        // вставка форматной строки в дерево DOM
        document.getElementById("info").innerHTML = out;
    }
</script>
```

Функции источника и потребителя определены, осталось вызвать их в сценарии файла приложения.

example_23. Вызов функций

```
<script>
    // получение данных курса с идентификатором = 2
```

```
    // данные сохраняем в переменную course
    let course = fnGetCourse(2);
    // форматный вывод в браузер
    // данные переменной course передаем в качестве параметра функции
    fnOutCourse(course);
</script>
```

Впрочем, логика приложения может быть еще более замысловатой. Все зависит от цели приложения, предметной области, опыта программиста.

Закрепляем пройденный материал. В **самостоятельных работах** выполним следующие задания:

- Создание функций источников данных.
- Создание функций потребителей данных.
- Обмен данными между функциями приложения.
- Ревьюирование логики работы приложений.

Ввиду возможности создания кода разными способами при выполнении самостоятельных заданий придерживайтесь следующих правил:

- Каждая функция выполняет не более одного действия. Действия форматирования и вставки контента можете рассматривать как одно или два, в зависимости от вашей логики программирования.
- Функции обмениваются между собой данными посредством передачи параметров и возврата значений.
- Предложенное решение может быть не оптимальным, и скорее рассматривает общий подход к организации функционального взаимодействия. Возможно, ваш код будет более удачен ;)

P.S.



Для отработки и закрепления учебного курса **донам** группы предоставляется следующий раздаточный материал.

К каждому уроку курса:

- Файлы **демонстрационного кода** (example);
- **Задачи** с решениями в контексте рассматриваемых вопросов урока (task).

К каждой теме курса:

- **Практические** работы.

